Android Performance + Security

Two interview-critical topics in one focused PDF · 2026 stack

Perf: cold start, Baseline Profiles, R8, frame timing, GC, ANR

Security: Keystore, cert pinning, biometric, Play Integrity

About this sheet: two distinct but related topics — performance and security are both about defending the user from your code. Performance defends them from waiting; security defends them from your bugs and from attackers. Same dense format as the other cheat sheets in this repo. Original prose, standard idiomatic patterns, interview-flavored.

PART 1 — Android Performance Engineering

1. Why performance — what it costs to be slow

METRIC	USER IMPACT	BUSINESS IMPACT
Cold start > 1.5 s	Users perceive the app as broken	~10% drop in d1 retention per second over 1 s
ANRs	"App not responding" dialog → force quit	Play Console flags; can block Play Store updates
Dropped frames (jank)	Scroll feels choppy → users blame your app, not the device	1-star reviews; loss of premium feel
Battery drain	Phone hot, dies fast → uninstall	System highlights battery-hogging apps in Settings
OOM crashes	App vanishes mid-flow	Counted in Crash-Free Sessions metric

Google's **Android Vitals** dashboard in Play Console tracks several of these. Cross thresholds and Play *literally hides your app from search results* in some categories. Performance isn't optional for shipping at scale.

2. The metrics that matter (what to measure)

METRIC	TARGET	TOOL
Cold start (TTID)	< 500 ms cold; < 200 ms warm	Macrobenchmark `StartupTimingMetric`
Time to first draw (TTFD)	Ideally same as TTID; aggressive target ≤ 1 s	Macrobenchmark
Frame timing (60 Hz target)	p99 < 16.6 ms; no dropped frames during scroll	Macrobenchmark `FrameTimingMetric` + Perfetto
Frame timing (120 Hz)	p99 < 8.3 ms	Same; check device's actual refresh rate
ANR rate	< 0.47% of daily users (Play threshold)	Play Console / Crashlytics
Crash-free sessions	≥ 99.5%	Play Console / Crashlytics
Heap memory	< 1/2 device class limit	Profiler memory tab
APK / Bundle size	App Bundle < 100 MB; install size < 50 MB	Android Studio APK Analyzer
Battery	Baseline: app shouldn't appear in Settings → Battery top consumers	Battery Historian + Perfetto

3. Cold start — phases & optimization

The phases (in order)

```
USER TAPS LAUNCHER ICON
        │
        ▼
[1] System Server: pick the right app process
        │
        ▼
[2] Zygote fork() → new process for your app
        │  (very fast; copy-on-write)
        ▼
[3] ART process init: load classpath, link JNI
        │
        ▼
[4] ContentProvider.onCreate (ALL providers in your manifest, sequentially)
        │  ← App Startup library batches these
        ▼
[5] Application.onCreate
        │
        ▼
[6] Activity.onCreate → setContent { ... }
        │
        ▼
[7] First Compose / View measure + layout + draw
        │
        ▼
[8] First frame submitted to display ← "Time To Initial Display" stops here
```

Where time goes (typical phone)

PHASE	TYPICAL COST	WHERE TO OPTIMIZE
Process fork (Zygote)	~50 ms	Out of your control — Android handles it
ContentProvider init	50-300 ms (per provider)	App Startup library (consolidate)
Application.onCreate	0-500 ms (depends on what you do)	Defer non-essential SDK init; parallelize
Activity.onCreate + first frame	200-800 ms	Lighter UI, Baseline Profile, defer heavy widgets

The cold-start optimization checklist

1. **Ship a Baseline Profile** — single biggest win for Compose apps (20-40% improvement).
2. **Use the Splash Screen API** (Android 12+) — covers the perceived gap with branded UI immediately.
3. **App Startup library** — replaces N hidden ContentProviders from libraries with one ordered initializer chain.
4. **Move non-essential work out of Application.onCreate**: analytics, push token registration, prefetch — defer to first idle or first user action.
5. **Lazy-init heavy singletons**: `by lazy { ... }` for OkHttpClient, Database — created on first use.
6. **Parallel SDK init** via `App Startup` dependencies graph; use a coroutine scope for I/O-bound init.
7. **R8 with shrinking** — smaller bytecode = faster classloading.

8. **Avoid disk I/O in onCreate** — DataStore, SharedPreferences are async; never block on them.
9. **Inflate less** — for Compose, simpler initial layout; for Views, avoid deep hierarchies.
10. **Defer images** — first frame doesn't need them; show placeholders, decode async.

4. Baseline Profiles — the single biggest perf win

A Baseline Profile is a list of "hot" methods (your startup paths and key user flows) that ART pre-AOT-compiles at install time. Skips JIT warm-up. Typical wins: cold start 20-40%, scroll jank 30-50% on covered flows. **Compose apps benefit most** — Compose runtime is method-heavy and JIT-warmup-prone.

Generate one

```
@RunWith(AndroidJUnit4::class)
class StartupBaselineProfile {
    @get:Rule val rule = BaselineProfileRule()

    @Test fun generate() = rule.collect(packageName = "com.app") {
        startActivityAndWait()
        device.findObject(By.text("Open Feed")).click()
        device.wait(Until.hasObject(By.text("Item 5")), 5_000)
        device.findObject(By.res("feed")).fling(Direction.DOWN)
    }
}
```

Run on a physical device. Output: `baseline-prof.txt` shipped in your APK.

How to verify it's working

- Run a Macrobenchmark with `StartupTimingMetric` twice — once with profile, once with `baselineProfile = false`. Compare.
- Check the APK contains `assets/dexopt/baseline.prof` (use APK Analyzer).
- Track Android Vitals after release; cold-start histogram should shift left.

> **Tip:** Play Console aggregates user profiles into a "Cloud Profile" automatically — but it takes weeks of usage. Ship your own Baseline Profile so day-1 users get fast starts.

5. R8 — shrink, optimize, obfuscate

R8 is Android's official code shrinker (replaces ProGuard since AGP 3.4). Three passes per release build:

1. **Shrink** — tree-shake from `-keep` roots; remove unreachable classes/methods/fields.
2. **Optimize** — inline single-use methods, propagate constants, devirtualize calls (single-impl interfaces become direct calls), eliminate dead branches.
3. **Obfuscate** — rename remaining classes/methods to short names (`com.app.repository.User` → `a.b.c`).

Result: 30-60% APK size reduction; faster classloading; smaller methods table.

Enable

```
android {
    buildTypes {
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
}
```

The `-keep` rules you actually need

```
# Reflection-based code R8 can't see used:
-keep class com.app.dto.** { *; }              # JSON DTOs (Gson/Moshi reflection)

# Keep enum values (R8 strips them by default):
-keepclassmembers enum * {
    public static **[] values();
    public static ** valueOf(java.lang.String);
}

# Keep classes with @Keep annotation
-keep @androidx.annotation.Keep class *
-keepclassmembers class * {
    @androidx.annotation.Keep *;
}

# Annotations stay (R8 might strip otherwise):
-keepattributes Signature, *Annotation*, EnclosingMethod, InnerClasses
```

> **Common bug:** reflection-based libraries (Gson, Moshi reflective, raw Retrofit annotations) silently break under R8 if you forget `-keep`. Always test the release build before shipping. Modern alternative: KSP-based serialization (kotlinx.serialization, Moshi-codegen) needs zero rules.

Upload `mapping.txt` to Play Console

R8 obfuscation makes stack traces unreadable. Upload `build/outputs/mapping/release/mapping.txt` to Play (Bundle Explorer → Deobfuscation files) — Play automatically deobfuscates crash traces. Same with Crashlytics.

6. Frame timing & jank — how Android renders frames

The frame loop (Choreographer + VSYNC)

The display sends VSYNC every **16.67 ms (60 Hz)** or **8.33 ms (120 Hz)**. Each VSYNC, Android's `Choreographer` dispatches three callbacks on the main thread — input → animation → traversal (measure + layout + draw). Anything that overruns the budget → dropped frame → user sees stutter.

```
VSYNC ──┬──────────┬──────────┬──────────┬──────────► time
        │          │          │          │
        │   Frame budget = 16.6 ms (60 Hz)
        ▼
   Choreographer dispatches:
     1. input events (touch, keys)
     2. animations (Animator + Compose recomposition)
     3. traversal (measure → layout → draw)

   ↓ submit display list to RenderThread
   ↓ RenderThread → GPU
   ↓ Surface flinger composites + presents at next VSYNC
```

What causes jank

CAUSE	SYMPTOM	FIX
Main-thread blocking I/O	Stutter, possibly ANR	`withContext(Dispatchers.IO)`
Large bitmap decode in onDraw	Scroll stutter on image-heavy lists	Decode async; use Coil/Glide; downsample at decode
Allocations in scroll loop	GC pauses every few seconds	Object pools, avoid lambda allocations, primitive State
Compose recomposition cascades	"Show recomposition counts" climbs absurdly	Stable parameters, defer State reads (see Compose cheat sheet)
Deep view hierarchies	Slow measure pass	ConstraintLayout, flat trees, Compose
Inflating layouts in onBindViewHolder	Scroll stutter on RecyclerView	One inflate per ViewType; mutate in onBind
Wide-color-gamut shaders	Render-thread time spikes on hi-end devices	Profile via Perfetto; simplify shaders

How to measure

```
@Test fun scrollPerf() = benchmarkRule.measureRepeated(
    packageName = "com.app",
    metrics = listOf(FrameTimingMetric()),
    iterations = 5,
    startupMode = StartupMode.WARM
) {
    pressHome(); startActivityAndWait()
    device.findObject(By.res("feed")).fling(Direction.DOWN)
}
// Output: frameDurationCpuMs (p50, p90, p99) — keep p99 under your refresh budget
```

7. Memory & GC — heap, native, leaks

The Android memory model

MEMORY REGION	WHAT LIVES THERE	LIMIT
Java heap	Kotlin/Java objects	Per-app cap (typically 256-768 MB by device class). `android:largeHeap` bumps it but is a smell.
Native heap	JNI allocations, Bitmap pixels (Android 8+), some Compose data	Bounded by overall device memory, not by Java heap cap
Stack	Per-thread frames + locals	~1 MB per thread
Code cache (ART)	JIT-compiled methods	Bounded by ART config
Graphics memory (GPU)	Hardware Bitmap pixels, surfaces	Bounded by device GPU memory

Android Concurrent Copying GC (ART, Android 8+)

- Generational + concurrent — most GC happens on a separate thread
- Stop-the-world pauses typically 1-5 ms (vs old Dalvik's 100+ ms)
- Copying collector eliminates fragmentation by moving live objects to a fresh region
- You still feel it during sustained allocation pressure (image-heavy scroll, animation)

How to be GC-friendly

- Avoid allocations in tight loops (scroll callbacks, onDraw, recomposition)
- Use object pools — BitmapPool (Glide/Coil), ViewHolder pool (RecyclerView)
- Prefer primitive arrays (`IntArray`) over `List<Int>` in hot paths
- Reuse `StringBuilder`; avoid string concatenation in loops
- Use Compose primitive State (`mutableIntStateOf`) over boxed (`mutableStateOf(0)`)
- Use `SparseArray`, `ArrayMap`, `ArraySet` for small collections (less per-entry overhead than HashMap)

Memory leaks — the killer

An object outlives its scope, holding references that prevent GC. The classic Android leaks:

- Static reference to Context / Activity
- Inner non-static class holding outer (Activity)
- Handler with delayed messages keeping Activity alive
- Coroutine launched from non-lifecycle scope (`GlobalScope`)
- Long-lived listener registered without being unregistered
- RxJava Disposable not disposed

Tool: LeakCanary. Auto-detects retained Activity/Fragment/ViewModel instances in debug builds. Free, drop-in, mandatory.

8. Bitmap memory — the biggest waste in image apps

CONFIG	BYTES PER PIXEL	NOTES
`ARGB_8888`	4	Default. Full color + alpha.
`RGB_565`	2	16-bit color, no alpha. Banding visible. Rare in 2026.
`HARDWARE` (API 26+)	~0 in Java heap	Pixels in GPU memory; can't read pixels on CPU. Default for image libraries.
`ALPHA_8`	1	Single-channel; for masks.

Math: a 4K image (3840×2160)

- ARGB_8888 = **33 MB** per bitmap
- Decoded full-size into a 200×200 ImageView = 32 MB wasted
- Solution: `BitmapFactory.Options.inSampleSize = 8` → decoded as 480×270 = 0.5 MB

Image library best practice

```
// Coil — automatically downsamples to target view size
AsyncImage(
    model = url,
    contentDescription = null,
    modifier = Modifier.size(240.dp)        // Coil decodes to ~240dp resolution
)
// Result: ~0.2 MB per visible thumbnail vs 33 MB raw decode.
```

Hardware bitmaps

Stored in GPU memory. Don't count toward Java heap. Drawn faster (no upload-to-GPU per draw). Trade-off: *can't read pixels* on CPU — copy/transform requires `bitmap.copy(ARGB_8888, isMutable = false)` first. Coil/Glide use HARDWARE by default on API 26+.

9. Battery optimization

What costs battery

ACTIVITY	COST
Cellular radio active (any data)	High — radio stays on for ~20 s "tail energy" after each request
GPS / fused location	Very high (PRIORITY_HIGH_ACCURACY = full GPS)
Wi-Fi scan / cellular scan	High
Wakelocks	Whatever was running drains baseline + work cost
Background Bluetooth scanning	Significant; restricted on Android 8+
Foreground Service	Whatever it does; survives Doze (privilege)
JIT compilation churn	CPU spikes on cold start (Baseline Profile mitigates)

The big rules

1. **Batch network calls.** Don't ping at random; coalesce + use HTTP/2 connection reuse.
2. **Use WorkManager constraints**: `NetworkType.UNMETERED`, `BatteryNotLow`, `RequiresCharging`.
3. **Location: pick the right priority**. `BALANCED_POWER_ACCURACY` for most, `HIGH_ACCURACY` only when actively navigating.
4. **Stop sensors in onPause**; never leak listeners.
5. **Avoid wakelocks**; if needed, hold for shortest possible window.
6. **Use FCM data messages** instead of polling for "did the server change?"
7. **Defer non-essential sync to charging time.**

Doze, App Standby, Restricted bucket

STATE	TRIGGERED BY	EFFECT
Doze	Device unplugged + screen off + stationary	Network deferred, alarms throttled, jobs deferred — until maintenance window
App Standby	App not used by user for days	Network access restricted; jobs throttled. Bucket: Active / Working set / Frequent / Rare / Restricted
Background restrictions (Android 9+)	User toggles via Settings, OR system auto-restricts heavy apps	App treated as Restricted bucket all the time

Design for these — they're not optional. WorkManager handles all of this automatically when you use constraints.

Tools

- **Battery Historian** — visualizes a Perfetto trace; shows wakelocks, network use, GPS sessions
- **Android Studio Energy Profiler** — live energy use during a session

- **Play Console: Excessive Wakelocks** + **Excessive Network Usage** dashboards — vitals

10. Network optimization

1. **Single shared OkHttpClient** — preserves connection pool + cache. Each `OkHttpClient.Builder().build()` creates a new pool.
2. **HTTP/2 by default** — OkHttp negotiates over TLS automatically. One TCP connection multiplexes many streams.
3. **Connection pool**: default keeps 5 idle connections for 5 min. Tune up if you hit many hosts.
4. **HTTP cache** — let OkHttp cache GET responses; respect server's `Cache-Control` + `ETag`.
5. **Batch requests** when possible (GraphQL, custom batch endpoint).
6. **Compression**: OkHttp adds `Accept-Encoding: gzip` automatically; enable Brotli on server for ~20% additional savings.
7. **Pre-fetch on Wi-Fi** when you have predictable next requests (next page of feed, next image).
8. **Avoid pre-flight CORS-equivalent overhead** (mostly server-side concern, but simple endpoints help).
9. **QUIC / HTTP/3** via Cronet for lossy mobile networks (10-20% TTFB improvement on 4G).

11. App Startup library — consolidate ContentProvider init

Many SDKs auto-initialize via a hidden `ContentProvider` in their manifest (Firebase, WorkManager, others). Each adds startup cost. App Startup library replaces them all with ONE provider that runs an ordered initializer chain.

```
// Manifest entry — opt out of a library's auto-init
<provider
    android:name="androidx.startup.InitializationProvider"
    android:authorities="${applicationId}.androidx-startup"
    tools:node="merge">
    <meta-data android:name="com.example.MyInitializer" android:value="androidx.startup" />
</provider>

// Your initializer
class MyInitializer : Initializer<MyLib> {
    override fun create(ctx: Context): MyLib = MyLib.init(ctx)
    override fun dependencies() = emptyList<Class<out Initializer<*>>>()
}
```

Why it matters: `Application.onCreate` is on the cold-start critical path; every ContentProvider runs *before* it. Reducing N providers to 1 cuts ~100+ ms typical.

12. ANR — what triggers it & how to prevent

What triggers ANR

- Main thread blocked > 5 s while user is interacting (touch, key)
- BroadcastReceiver doesn't return from `onReceive` in 10 s (foreground) / 60 s (background)
- Service doesn't return in time (20 s for foreground)
- JobService / WorkManager exceeds 10 min wall clock without setForeground

The killer is always one of these

1. Synchronous DB query / file read on main thread
2. Synchronous network call
3. Large JSON parsing (~100 KB+ Gson reflection on main)
4. Image decode without downsampling
5. Deadlocks in coroutines / Mutex
6. Choreographer doing too much per frame (deep View hierarchies, complex Compose modifiers)

Prevention

- **StrictMode** in debug — catches main-thread disk + network access; will crash dev builds, surface bugs early
- **Always** `withContext(Dispatchers.IO)` for any blocking I/O
- **Profile release builds with Macrobenchmark**; ANRs are quietly logged in Play Console
- **Watchdog ANR detection** in your own code (post a heartbeat to main; if not echoed in 5 s, capture all thread stacks for telemetry)
- **Use Coroutines instead of Threads** — much harder to deadlock with structured concurrency

13. Compose-specific performance

Compose has its own performance discipline. Quick reference (full treatment in the Compose cheat sheet):

- **Stable parameters** — `ImmutableList` over `List`, `@Immutable` on data classes
- **Defer State reads to lower phases** — `Modifier.offset { value }` reads in Layout; `graphicsLayer { rotationZ = v }` reads in Drawing
- **Stable list keys** — entity ID, never index
- **Memoize lambdas** with stable captures — the compiler usually does this for you
- **Avoid passing** `State<T>` **as parameter** — read at caller, pass value
- **Compose Compiler reports** are the ground truth — every Composable taking parameters should be `skippable`
- **Layout Inspector recomposition counts** — interactively find recomp hotspots

14. Profiling tools — what each is for

TOOL	WHEN TO USE
Android Studio Profiler	Live CPU / memory / energy / network during interactive session. Best for "what's allocating right now"
Macrobenchmark	Reproducible end-to-end perf measurements on release builds. CI-friendly. Use for cold start, scroll, custom flows.
Perfetto / Systrace	Low-level kernel + user-space trace. Best for understanding scheduling, frame timing internals, cross-thread contention.
Layout Inspector	Live View hierarchy + Compose recomposition counts. Best for "why is this recomposing".
Database Inspector	Live Room queries; check for N+1 queries, slow JOINs.
Network Inspector	OkHttp request/response timeline. Best for "is my caching working" / "are headers right"
LeakCanary	Auto-detect retained Activity/ViewModel/Fragment instances in debug.
Compose Compiler reports	Per-Composable skippable / restartable status. Static analysis of stability.
APK Analyzer	What's bloating my APK? Per-DEX class size, resource size, native libs.
Play Console Vitals	Real-user metrics. The truth about whether your app is fast for actual users.

15. Performance shipping checklist

BEFORE EVERY RELEASE

- R8 minification + resource shrinking enabled
- Baseline Profile generated and shipped in APK
- Splash Screen API integrated (Android 12+)
- App Startup library consolidates SDK init
- App Bundle (.aab) ships, not APK
- WebP / AVIF for raster images; VectorDrawable for icons
- WorkManager constraints on all background sync
- Macrobenchmark cold-start & scroll tests pass thresholds
- LeakCanary clean in debug
- Compose Compiler reports show 90%+ skippable
- Compose stability metrics archived for diff next release
- Mapping.txt uploaded to Crashlytics + Play

COMMON MISTAKES

- Profiling debug builds (misleading)
- Heavy SDK init in Application.onCreate
- One ViewModel per leaf Composable
- `List<T>` as Composable parameters (unstable)
- Reading `scrollState.value` in Composition
- HARDWARE bitmap copied to ARGB_8888 unintentionally (CPU read)
- Animating with `Modifier.rotate` instead of `graphicsLayer`
- Using `android:largeHeap` instead of fixing the leak
- GlobalScope coroutines (no cancellation, no scoping)
- Inflate-in-onBindViewHolder anti-pattern

PART 2 — Android Security

16. Threat model — what you're defending against

THREAT	WHAT IT LOOKS LIKE	PRIMARY DEFENSE
Token / credential theft from device	Rooted phone, malware reading shared storage	Keystore-backed encryption + EncryptedSharedPreferences
Man-in-the-middle (MITM) on network	Hostile Wi-Fi, untrusted CA installed	Certificate pinning + Network Security Config
Unauthorized API access	Reverse-engineered app + replayed requests	Server-side auth + Play Integrity API
Reverse engineering of business logic	APK extraction, decompilation	R8 obfuscation + native code for sensitive logic
Tampered APK	Sideloaded modified APK	Play App Signing + Play Integrity
Insecure data sharing with other apps	Implicit intents leaking PII	FileProvider + explicit intents + signature permissions
Phishing via deep links	Malicious app catches your `myapp://` link	App Links with `autoVerify` + assetlinks.json
WebView XSS	JavaScript bridge abuse	Disable JS unless needed; lock `JavascriptInterface` tightly

Security is layered defense. None of these alone is enough — chain them.

17. Android Keystore — the foundation

The Android Keystore is a system service that generates, stores, and uses cryptographic keys. On capable devices, keys are hardware-backed via the TEE (Trusted Execution Environment) or StrongBox — meaning they **cannot be exfiltrated** even on a rooted device.

Generate a key

```
val spec = KeyGenParameterSpec.Builder("my_key_alias",
        KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT)
    .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
    .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
    .setKeySize(256)
    .setUserAuthenticationRequired(false)         // or true for biometric-gated key
    .setIsStrongBoxBacked(true)                    // hardware-backed if available
    .build()

val generator = KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES,
                                         "AndroidKeyStore")
generator.init(spec)
generator.generateKey()
```

Encrypt with the key

```
val ks = KeyStore.getInstance("AndroidKeyStore").apply { load(null) }
val key = ks.getKey("my_key_alias", null) as SecretKey

val cipher = Cipher.getInstance("AES/GCM/NoPadding")
cipher.init(Cipher.ENCRYPT_MODE, key)
val iv = cipher.iv
val ciphertext = cipher.doFinal(plaintext)
// Persist (iv, ciphertext); decrypt later with same key + iv
```

Key features

- Keys are referenced by **alias**; the key bytes never enter your process memory on hardware-backed devices
- Set `setUserAuthenticationRequired(true)` to require biometric/PIN before the key can be used — gates sensitive operations
- StrongBox (API 28+, Pixel 3+) gives an extra hardware tier — defends against TEE compromise
- Keys can be **attested** — server can verify the key really lives in TEE/StrongBox via remote attestation

18. Storing secrets — EncryptedSharedPreferences & alternatives

Most apps need to persist a few sensitive blobs (refresh tokens, encrypted credentials, OAuth state). Don't put these in plain SharedPreferences.

EncryptedSharedPreferences (Jetpack Security — but in 2026, deprecated for new code)

```
val mainKey = MasterKey.Builder(ctx)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .build()

val prefs = EncryptedSharedPreferences.create(
    ctx, "secure_prefs", mainKey,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
)
prefs.edit().putString("refresh_token", token).apply()
```

> **2026 status:** Jetpack Security (which provides `EncryptedSharedPreferences` and `EncryptedFile`) was **deprecated in 2024** with no direct replacement. Existing usage still works. Long-term, prefer **DataStore + your own Keystore-backed encryption**, or **Google Tink** (which underlies Jetpack Security anyway).

Recommended modern approach: DataStore + manual Keystore-backed encryption

```
class SecureDataStore(val ctx: Context) {
    private val dataStore = ctx.preferencesDataStore("secure")
    private val crypto = KeystoreAesGcm("app_data_key")

    suspend fun putString(key: String, value: String) {
        val encrypted = crypto.encrypt(value.toByteArray())
        dataStore.edit { it[stringPreferencesKey(key)] = Base64.encode(encrypted) }
    }

    suspend fun getString(key: String): String? {
        val encoded = dataStore.data.first()[stringPreferencesKey(key)] ?: return null
        return crypto.decrypt(Base64.decode(encoded)).toString(Charsets.UTF_8)
    }
}
```

For full-file encryption

`EncryptedFile` still works (deprecated alongside ESP). Modern alternative: encrypt with Keystore-backed AES-GCM yourself, write ciphertext via `OutputStream`.

Common mistakes

- **API keys in BuildConfig** — trivially extractable from APK. Move to a backend gateway; client never sees them.
- **Hardcoded encryption keys** in code/strings — same as plaintext. Always use Keystore.
- **Storing PINs / passwords** — never; store a **hash** (Argon2id) for verification only.
- **Logging tokens** — Logcat persists; tokens leak via bug reports. Strip in proguard or use a logging framework with redaction.

19. Network Security Config — TLS, cleartext, trust anchors

An XML file in your app that controls how Android validates HTTPS connections, what cleartext is allowed, which CAs are trusted.

```xml
<!-- res/xml/network_security_config.xml -->
<network-security-config>
    <base-config cleartextTrafficPermitted="false">
        <trust-anchors>
            <certificates src="system" />
            <!-- DO NOT include user CAs in production -->
        </trust-anchors>
    </base-config>

    <!-- Pin specific domains -->
    <domain-config>
        <domain includeSubdomains="true">api.example.com</domain>
        <pin-set expiration="2026-12-31">
            <pin digest="SHA-256">AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=</pin>
            <pin digest="SHA-256">BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB=</pin> <!-- backup -->
        </pin-set>
    </domain-config>

    <!-- Allow cleartext only for debug builds (use a debug-overrides config) -->
    <debug-overrides>
        <trust-anchors>
            <certificates src="user" />
            <!-- Charles/Proxyman cert can intercept HTTPS in debug only -->
        </trust-anchors>
    </debug-overrides>
</network-security-config>
```

Apply via manifest: `android:networkSecurityConfig="@xml/network_security_config"`.

What this defends against

- **cleartextTrafficPermitted=false** — accidental HTTP usage will fail (forces HTTPS for all calls)
- **Excluding `user` trust anchors in prod** — malicious user-installed CAs (sometimes installed by enterprise MDM, sometimes by malware) can't intercept your TLS
- **Pin-set** — even a real CA being compromised can't make a hostile cert your app trusts

20. Certificate pinning — the deeper alternative to NSC pinning

NSC pinning above works at the system level. OkHttp's `CertificatePinner` works at the library level — same effect, more programmatic control:

```
val pinner = CertificatePinner.Builder()
    .add("api.example.com", "sha256/AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=")
    .add("api.example.com", "sha256/BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB=")   // backup
    .build()

val client = OkHttpClient.Builder().certificatePinner(pinner).build()
```

Critical: ALWAYS ship at least one backup pin

If your prod cert rotates and you forgot to ship an update, the app breaks for users — they can't reach your API. Backup pins for the next-issued cert (or its intermediate) are insurance.

Plan for cert lifecycle

- Pin both **leaf + intermediate**, OR pin to your CA's intermediate (less rotation churn)
- Maintain a **remote killswitch** — server-side flag that disables pinning if shipped pins are wrong
- Monitor pin failures via analytics — sudden spike = your cert rotated and you didn't update the app

How to extract a pin

```
# Get cert from server
openssl s_client -servername api.example.com -connect api.example.com:443 \
  | openssl x509 -pubkey -noout \
  | openssl pkey -pubin -outform der \
  | openssl dgst -sha256 -binary \
  | openssl enc -base64
```

21. Biometric authentication

Use the Jetpack `BiometricPrompt` API. Two modes: device-credential (PIN/pattern fallback OK) and biometric-only. Bind sensitive operations to authentication via a Keystore key with `setUserAuthenticationRequired(true)`.

Simple biometric prompt

```
val prompt = BiometricPrompt(activity, executor,
    object : BiometricPrompt.AuthenticationCallback() {
        override fun onAuthenticationSucceeded(r: BiometricPrompt.AuthenticationResult) {
            unlockUserData()
        }
        override fun onAuthenticationError(code: Int, msg: CharSequence) { ... }
    })

val info = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Confirm it's you")
    .setSubtitle("Authenticate to view your wallet")
    .setAllowedAuthenticators(BIOMETRIC_STRONG or DEVICE_CREDENTIAL)
    .build()

prompt.authenticate(info)
```

For high-stakes ops — bind to a CryptoObject

For payments, "delete account", reveal secret data, you want the auth to *materialize* a Keystore key (not just gate UI):

```
val key: SecretKey = ks.getKey("payment_key", null) as SecretKey
val cipher = Cipher.getInstance("AES/GCM/NoPadding").apply {
    init(Cipher.ENCRYPT_MODE, key)
}
prompt.authenticate(info, BiometricPrompt.CryptoObject(cipher))

// On success: callback.result.cryptoObject.cipher is now usable
// The key was Keystore-bound to setUserAuthenticationRequired(true)
// → without auth, the cipher would have thrown.
```

AUTHENTICATOR CLASS	EXAMPLES	CAN GATE KEYSTORE KEY?
BIOMETRIC_STRONG (Class 3)	Real fingerprint, hardware face ID	Yes
BIOMETRIC_WEAK (Class 2)	Some face unlock implementations	No (too spoofable)
DEVICE_CREDENTIAL	PIN, pattern, password	Yes (Android 11+)

22. Play Integrity API — anti-tamper / anti-bot

Google's API that returns a signed verdict telling your backend whether the request comes from a genuine, unmodified app on a Play-protected device. Replaces SafetyNet (deprecated 2024).

```
val integrity = IntegrityManagerFactory.create(ctx)
val nonce = serverProvidedNonce       // fresh, server-generated, per request
val req = IntegrityTokenRequest.builder()
    .setNonce(nonce)
    .build()
val token = integrity.requestIntegrityToken(req).await().token()
api.submit(token)                          // server validates with Play Integrity
```

Server validates and gets back a verdict

FIELD	MEANING
`appIntegrity.appRecognitionVerdict`	PLAY_RECOGNIZED / UNRECOGNIZED_VERSION / UNEVALUATED
`deviceIntegrity.deviceRecognitionVerdict`	MEETS_BASIC / MEETS_DEVICE / MEETS_STRONG / (none)
`accountDetails.appLicensingVerdict`	LICENSED / UNLICENSED

When to use

- Account creation / signup (defeat bot farms)
- Payment-related operations (defeat replay attacks)
- Reward claims, in-app currency operations
- Sensitive APIs where you don't want a reverse-engineered client

> **Quota:** Play Integrity has free tier limits (a few thousand requests/day). For high-volume signup flows you may need Standard or Classic tier or to reserve calls for risky users only.

23. WebView security

WebView is an embedded browser. If you embed remote content, you're embedding a security boundary. Two big rules:

1. **Don't enable JavaScript unless you absolutely need it.**
2. **Never expose unsafe APIs via `JavascriptInterface`.**

Lock-down configuration

```
WebView(ctx).apply {
    settings.javaScriptEnabled = true          // only if needed
    settings.domStorageEnabled = true
    settings.allowFileAccess = false
    settings.allowContentAccess = false
    settings.allowFileAccessFromFileURLs = false          // XSS goldmine if true
    settings.allowUniversalAccessFromFileURLs = false
    settings.mixedContentMode = WebSettings.MIXED_CONTENT_NEVER_ALLOW

    addJavascriptInterface(SafeBridge(), "AndroidApp")   // validate every method!
    webViewClient = WebViewClient()                       // don't allow URL navigation outside whitelist
}
```

For loading external URLs — prefer Custom Tabs

```
val tabs = CustomTabsIntent.Builder()
    .setShowTitle(true)
    .setToolbarColor(MaterialColors.getColor(...))
    .build()
tabs.launchUrl(ctx, Uri.parse("https://example.com"))
```

Custom Tabs use the system Chrome with your branding — gets all of Chrome's security model, no embedded browser to worry about.

24. Storage security & runtime permissions

Scoped Storage (Android 11+)

- Apps can't access arbitrary external storage paths
- Use `MediaStore` for photos/videos/audio (with appropriate `READ_MEDIA_*` permission since 13)
- Use **Photo Picker** (Android 13+) for photo selection — no permission required
- Use `FileProvider` for sharing files with other apps
- App-private files: `context.filesDir`, `getExternalFilesDir()` — wiped on uninstall

Permission rules

- **POST_NOTIFICATIONS** (Android 13+): runtime perm; ask only when first useful
- **READ_MEDIA_IMAGES / VIDEO / AUDIO** (Android 13+) replace READ_EXTERNAL_STORAGE
- **READ_MEDIA_VISUAL_USER_SELECTED** (Android 14+): partial photo access
- **USE_FULL_SCREEN_INTENT** (Android 14+): user-grantable; required for incoming-call style notifs
- **SCHEDULE_EXACT_ALARM**: only for time-critical use cases

25. Deep link security

Custom-scheme deep links (`myapp://x`) can be hijacked by any other app declaring the same scheme. App Links (`https://app.example.com/x`) verified via Digital Asset Links can't be hijacked.

```
<!-- AndroidManifest.xml -->
<activity android:name=".DealActivity">
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="https" android:host="app.example.com"
              android:pathPrefix="/deal/" />
    </intent-filter>
</activity>
```

Host `https://app.example.com/.well-known/assetlinks.json`:

```
[{
  "relation": ["delegate_permission/common.handle_all_urls"],
  "target": {
    "namespace": "android_app",
    "package_name": "com.example.notes",
    "sha256_cert_fingerprints": ["AB:CD:..."]
  }
}]
```

Now Android verifies your app owns that domain at install time — no other app can claim those links.

Validate deep link payloads

Even App Links need server-side validation. Don't trust IDs in deep links — verify on backend that the user has access to the deep-linked resource.

26. Obfuscation strategy & build-time hardening

- **R8 with default optimize ProGuard rules** — handles 80% of casual reverse engineering
- **Move highly sensitive logic to native (NDK)** — disassembling C++/ARM is significantly harder than Kotlin → Smali
- **Tink / native cipher implementations** over hand-rolled crypto — avoid timing attacks, padding oracle bugs
- **Disable backups for sensitive data**: `android:allowBackup="false"` in manifest, or per-file backup rules
- **Disable debug builds in production manifest**: `android:debuggable="false"` (set automatically by release builds)
- **Use Play App Signing** (mandatory for new apps) — Play holds the signing key; you upload signed with your upload key. Lower compromise blast radius.
- **Detect rooted devices** via Play Integrity (deviceIntegrity verdict) — react in policy, not just in client code

- **Detect tampering** via signature verification on startup (cheap, deters casual modders)

27. Security shipping checklist

BEFORE EVERY RELEASE

- Network Security Config in place; cleartext disabled
- Certificate pinning with at least 1 backup pin
- Refresh token in EncryptedSharedPreferences / Keystore-backed encryption
- R8 minify + obfuscate enabled
- App Bundle signed via Play App Signing
- POST_NOTIFICATIONS asked at the right moment (not on app start)
- WebView locked down (or replaced with Custom Tabs)
- App Links autoVerify + assetlinks.json hosted
- Sensitive log lines stripped (no tokens / PII in Logcat)
- Play Integrity gating sensitive flows
- Biometric prompt + key gating for payment / wallet
- android:allowBackup=false (or backup rules excluding secrets)

COMMON SECURITY MISTAKES

- API keys in BuildConfig (extractable)
- Hardcoded encryption keys in code
- SharedPreferences with PII in plaintext
- WebView with JS enabled + JavascriptInterface exposing app methods
- Custom-scheme deep links (no autoverify)
- No backup pin → app breaks on cert rotation
- Trusting deep-link payload IDs without server-side check
- Debug-only NSC (with user CAs) shipped to prod
- android:exported=true on every Activity (broaden attack surface)
- Forgetting to upload mapping.txt (unreadable crash traces)
- Hand-rolled crypto (broken IVs, ECB mode, weak random)
- Logging full request/response in production builds

28. Interview Q&A — perf + security

Q: "Walk me through how you'd reduce cold start time by 30%."

Three layers. (1) **Baseline Profile** — pre-AOT compile startup paths; biggest win on Compose apps. (2) **App Startup library** — consolidate hidden ContentProviders from libraries into one ordered chain; saves ~100 ms of pre-Application setup. (3) **Defer non-essential SDK init** from Application.onCreate; lazy-init heavy singletons (Database, OkHttpClient). (4) **R8 with shrinking** — smaller bytecode = faster classloading. Verify each via Macrobenchmark with `StartupTimingMetric`.

Q: "Why do Compose apps benefit from Baseline Profiles disproportionately?"

Compose runtime is method-heavy and JIT-warmup-prone — composition machinery dispatches through many tiny generated functions. JIT takes seconds to identify them as hot and AOT-compile. A Baseline Profile pre-

compiles those methods at install time, so the first user interaction has them already AOT'd. Typical wins: cold start 20-40%, scroll jank 30-50%.

Q: "How do you find an unnecessary recomposition?"

Two-pronged. **Static**: enable Compose Compiler metrics, check generated `*-composables.txt` — every Composable taking parameters should be `skippable`. If not, find the unstable parameter (usually `List<T>`) and switch to `ImmutableList`. **Dynamic**: Layout Inspector → "Show recomposition counts" — interact with the screen, find Composables with absurdly high counts, fix per Compose perf rules (defer State reads, use derivedStateOf, hoist unstable lambdas).

Q: "Difference between R8 and ProGuard?"

R8 is Google's official replacement for ProGuard since AGP 3.4. Same general purpose (shrink + optimize + obfuscate) but compiled into AGP, faster, integrates with D8 dex compilation, supports Kotlin metadata better, has a more aggressive optimizer. ProGuard rules (`proguard-rules.pro`) are still consumed — backwards compatible.

Q: "How do you store a refresh token securely?"

EncryptedSharedPreferences if you have to use the deprecated Jetpack Security artifact, or DataStore + manual Keystore-backed AES-GCM encryption (preferred for new code). Key generated in Android Keystore (hardware-backed via TEE/StrongBox where available — keys can't be exfiltrated even on rooted devices). For high-stakes operations, gate the key with `setUserAuthenticationRequired(true)` so biometric/PIN is required before use.

Q: "Certificate pinning — how, and what's the gotcha?"

Configure via Network Security Config (declarative XML) or OkHttp's `CertificatePinner` (programmatic). Pin SHA-256 of the SPKI of an intermediate or leaf cert. **Gotcha:** always ship at least one backup pin. If your prod cert rotates and you didn't update the app, every install breaks. Plan for cert lifecycle and have a remote killswitch.

Q: "What is Play Integrity and when do you use it?"

Anti-tamper / anti-bot API; replaced SafetyNet in 2024. Returns a signed verdict telling your backend whether the request comes from a genuine, unmodified app on a Play-protected device. Three verdicts: app integrity (genuine APK?), device integrity (basic / device / strong), license (Play-licensed user). Use for: signup, payment, account-related ops, sensitive APIs. Free tier has quota — call sparingly.

Q: "Why is GlobalScope a perf and security concern?"

Perf: leaks coroutines (no parent → never cancelled). Hidden allocations accumulate; tests can't drain. Security: Coroutines launched from GlobalScope might outlive the user's session, doing work after logout — sending stale auth tokens, logging stale state. Use lifecycle-bound scopes (`viewModelScope`, `lifecycleScope`) or a properly-cancelled custom `CoroutineScope(SupervisorJob() + ...)`.

Q: "How do you find an ANR cause?"

Three places. (1) Play Console Vitals shows ANR frequency + the main thread stack at time of ANR. (2) Crashlytics same data. (3) Reproduce locally with StrictMode enabled (catches main-thread I/O). Common causes: synchronous DB on main, large JSON parse, bitmap decode, deadlock in coroutines. Fix: `withContext(Dispatchers.IO)`, async parse, downsample at decode.

Q: "How do you avoid memory leaks?"

Run LeakCanary in debug — instant alerts on retained Activity/ViewModel/Fragment. Common fixes: avoid static references to Context/Activity; replace inner classes with static nested + WeakReference; cancel coroutines via lifecycle scopes; unregister listeners in onPause/onStop; use `applicationContext` in long-lived components.

Q: "Compose stability - what does it mean and why does it matter?"

A type is "stable" if Compose can prove its `equals()` is honest and the type doesn't mutate without notification. Stable + unchanged params → Composable can skip on recomposition. Unstable params → conservative recompose every time. Most production wins come from making types stable: use `ImmutableList`, `val`-only data classes, `@Immutable` annotations.

29. One-page recall card

Performance — measure these

- Cold start: Macrobenchmark StartupTimingMetric, target < 500 ms
- Frame timing: FrameTimingMetric, target p99 < 16.6 ms
- ANR rate: < 0.47% (Play threshold)
- Crash-free sessions: ≥ 99.5%

Performance — biggest levers

- Baseline Profile (cold start +20-40%, scroll +30-50%)
- R8 minify + shrink (APK -30-60%)
- App Startup library (consolidate ContentProviders)
- Dispatchers.IO for blocking, Default for CPU
- StrictMode in debug, never main-thread I/O in prod
- Compose: stable params, defer reads to lower phases, derivedStateOf
- Coil/Glide HARDWARE bitmap + downsample
- WorkManager constraints (UNMETERED, BatteryNotLow)

Security — must-have layers

- Network Security Config + cert pinning + 1 backup pin
- Keystore-backed encryption for refresh tokens
- BiometricPrompt + CryptoObject for high-stakes ops
- Play Integrity for signup / payment endpoints
- R8 obfuscation; mapping.txt to Crashlytics + Play
- App Links + autoVerify + assetlinks.json
- Custom Tabs over WebView for external URLs
- Scoped Storage / Photo Picker over READ_EXTERNAL_STORAGE
- android:allowBackup=false (or per-file rules)
- Play App Signing (mandatory anyway)

Avoid (perf + security)

- API keys in BuildConfig
- SharedPreferences with PII plaintext

- WebView with JS + JavascriptInterface exposing methods
- GlobalScope coroutines
- android:largeHeap (mask for leak)
- Profiling debug builds for perf
- Hardcoded encryption keys
- Pinning without backup pins
- Heavy work in Application.onCreate

Read top to bottom for understanding · skim §15 (perf checklist), §27 (security checklist), §28 (Q&A), §29 (recall) before any interview. Pair with the other cheat sheets in this repo.